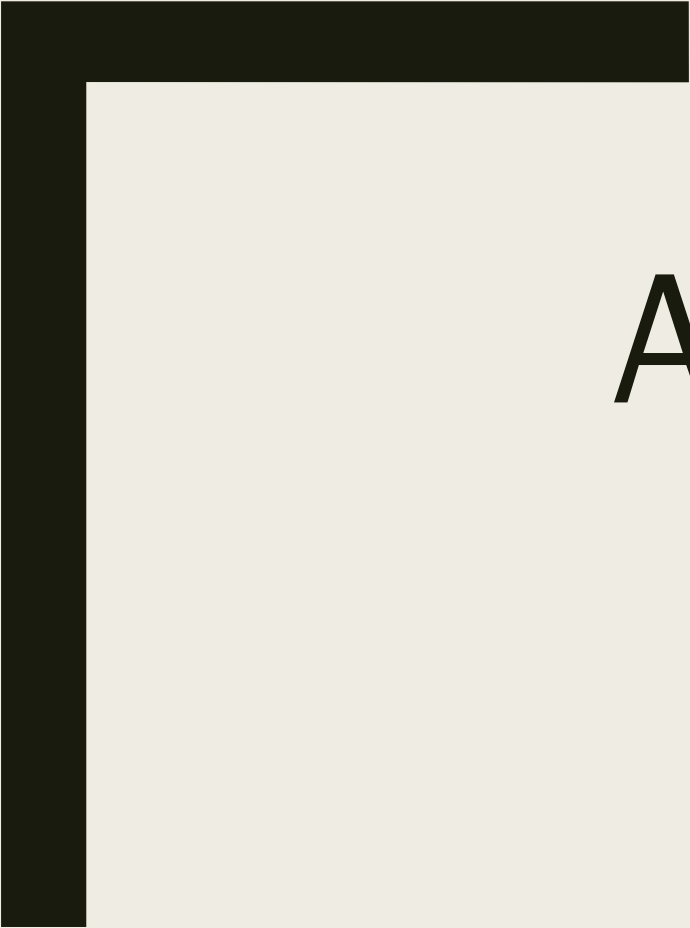
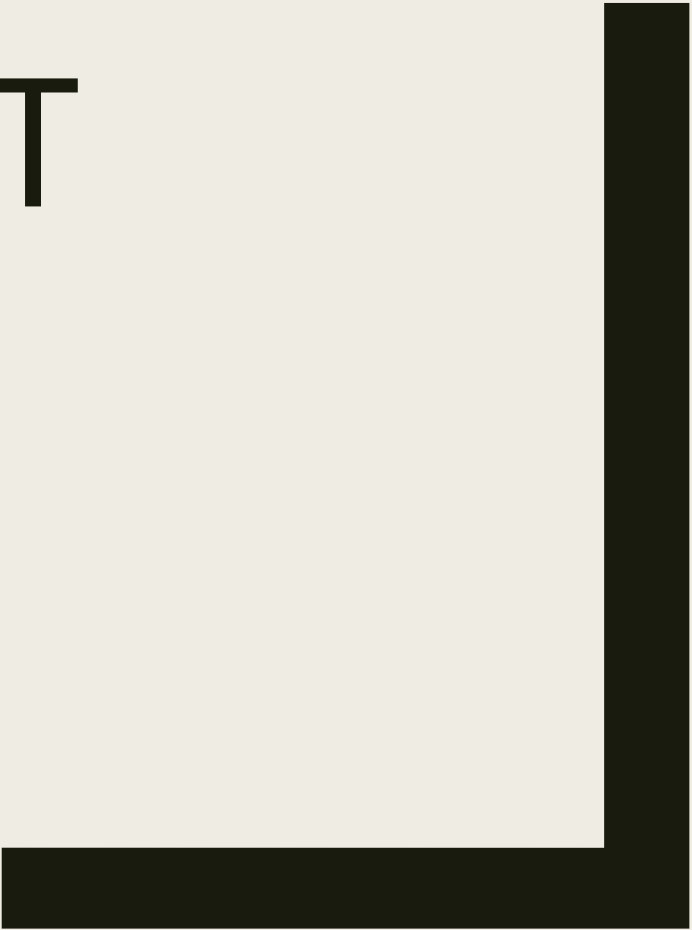




AMR JETBOT TUTORIAL



Jetbot Intro

- The Jason Nano is an official development board from Nvidia. Unlike the Raspberry Pi, it features a GPU, allowing it to run various deep learning applications without being limited by hardware performance. JetBot is an artificial intelligence robot kit developed based on the Jetson Nano.



Course Outline: Week 1

Lesson 1: JetBot Introduction and Basic Control

Duration: 2 hours Objective: Understand the composition, basic functions, and environment configuration of JetBot, and learn to operate JetBot using Python.

Outline :

1. **JetBot Intro** (20 minute)
 1. Introduces JetBot's hardware components: Jetson Nano, dual drive motors, camera, Wi-Fi module, etc.
 2. Overview of AI applications and JetBot's uses (such as object tracking and autonomous navigation).
2. **JetBot Development environment setup** (30 minute)
 1. How to start JetBot, connect to Wi-Fi, and access Jupyter Notebook.
 2. Introduction to using Jupyter Notebook
3. **Basic Programming and Motor Control**(50 minute)
 1. Teach students how to use Python to control JetBot's motors, including forward, backward, and turning movements.
 2. Have students try simple movement commands and observe JetBot's behavior.
4. **Classroom exercises and discussions**(20 minute)
 1. Students try to write basic motion control programs on Notebooks and conduct small-scale motion tests.

Course Outline: Week 2

Lesson 2: JetBot Perception and Image Processing

Duration: 2 hours

Objective: Understand JetBot's perception system and learn how to use the camera for image processing and object detection.

Outline :

1. **Introduction to JetBot's Camera Function** (15 minute)
 1. Describes how cameras are integrated with JetBot for perception.
 2. Explains the importance of cameras in site navigation (e.g., obstacle detection and path tracking).
2. **OpenCV Basics and Image Processing** (45 minute)
 1. Introduces the basic functions of the OpenCV image processing library and how to use it in Jupyter Notebook.
 2. Use OpenCV to perform basic image reading, display, and simple image processing (such as grayscale conversion and edge detection). °
3. **Object detection and tracking** (45 minute)
 1. Students will learn how to use the JetBot's camera to detect objects and write a tracking program using Python.
 2. Demonstrates how to enable JetBot to perform simple tracking based on objects detected by the camera.
4. **Exercises** (15 minute)
 1. Program JetBot to detect and track objects of a specific color °

Course Outline: Week 3

Lesson 3: Site Navigation and Autonomous Navigation

Time: 2 hours

Objective: Combine the knowledge from the first two lessons to implement JetBot's autonomous navigation capabilities. °

Outline :

1. **Path Planning and Navigation Basics** (30 minute)
 1. Introduces the basic concepts of path planning and navigation.
 2. Helps students understand how to use cameras and sensors to perceive the environment.
2. **Site design and goal setting**(30 minute)
 1. Set up a cruising area within the classroom, which may include obstacles, turning areas, etc.
 2. Set a cruising goal, such as moving along a specific path or avoiding obstacles
3. **Achieve autonomous cruising** (45 minute)
 1. Have students to write code to control the JetBot to autonomously navigate a field, avoiding obstacles and tracking a path based on camera input.
 2. Test different navigation strategies and adjust the code based on the results.
4. **Final Practice Exercises** (15 minute)
 1. Demonstrate the results of JetBot's navigation within the venue and discuss challenges encountered and solutions. °

Assembling Jetbot

1. Jetson Nano
2. Micro SD card
3. JetBot chassis and base
4. Lens holder
5. Acrylic sheet
6. JetBot expansion board
7. IMX218-160
8. Wireless network card
9. DC motors x 2
10. Wheels
11. Universal casters
12. 18650 batteries x 3
13. 12.6V power cable
14. Wireless remote control handle
15. Phillips screwdriver
16. Flex cable
17. Screw kit
18. Wrench
19. Fan (This article includes a separate fan installation)

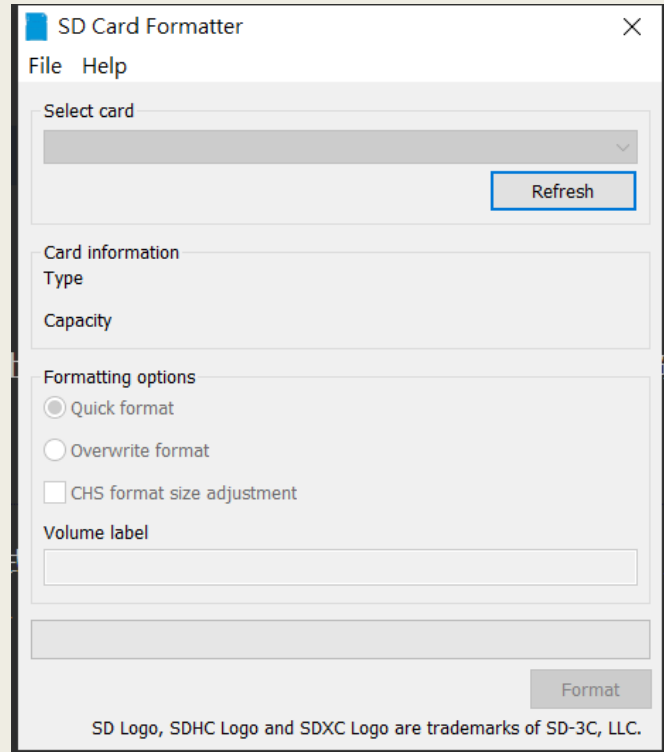


<https://www.youtube.com/watch?v=jGna0oGKdV4>

Environmental Installation

- Format the prepared Micro SD card.
- Flash the system to the Micro SD card.
- Power on the system to ensure it is functioning normally.

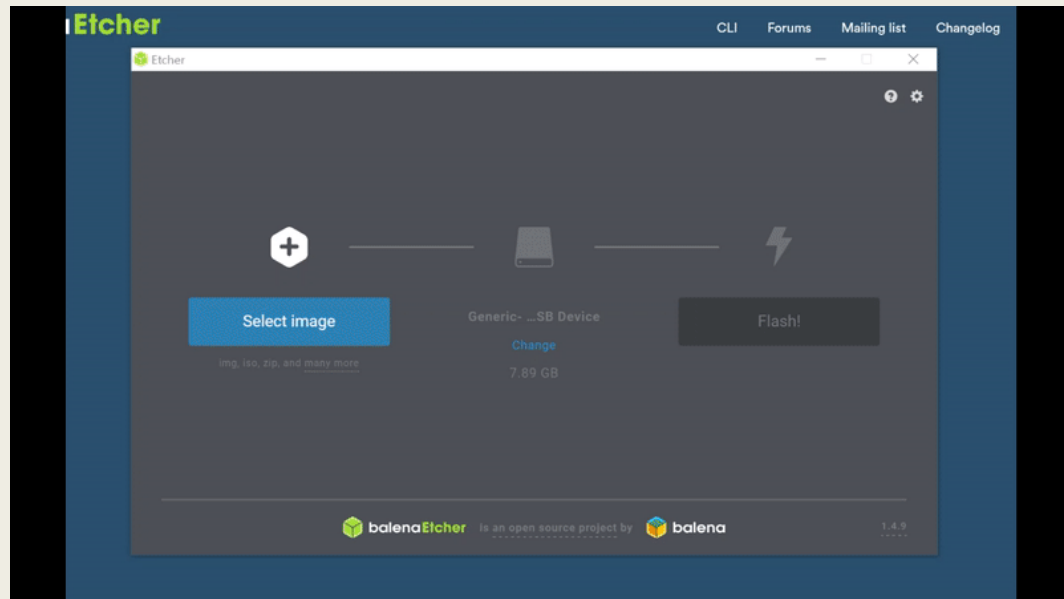
Format the prepared Micro SD Card



Recommend using SD Card Formatter.
First, insert the SD card into your computer
and click Quick Format.

Burn the system into the Micro SD Card

1. Download the official image: https://jetbot.org/master/software_setup/sd_card.html
2. Use a flashing software to burn the image to the SD card. Etcher is recommended.
3. Insert the SD card into the Jason Nano.



Latest Release

Platform	JetPack Version	JetBot Version	Download	MD5 Checksum
Jetson Nano 2GB	4.5	0.4.3	jetbot-043_nano-2gb-jp45.zip	e6dda4d13b1b1b31f648402b9b742152
Jetson Nano (4GB)	4.5	0.4.3	jetbot-043_nano-4gb-jp45.zip	760b1885646bfad8590633acca014289

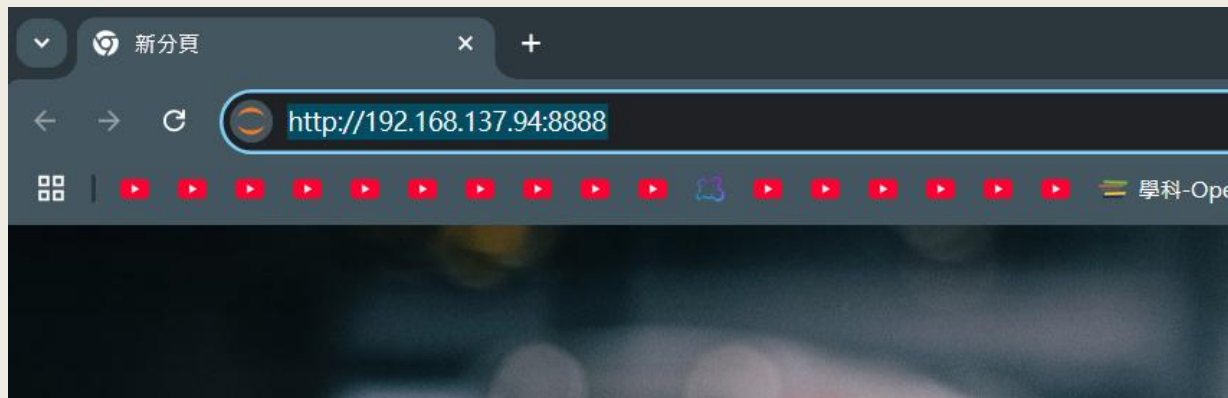
Power on and make sure the system is functioning normally.

When you first boot up, you need to connect the HDMI cable to the monitor and prepare a USB keyboard.

- Initially, the password to log in to JetBot is: jetbot.
- When the terminal appears on the screen, use the command: `sudo nmcli dev wifi connect (wifi name) password (wifi password) ifname wlan0`.
- Once connected to wifi, you can unplug the HDMI cable and keyboard.
- Then reboot the computer.

Remotely connect to Jetbot

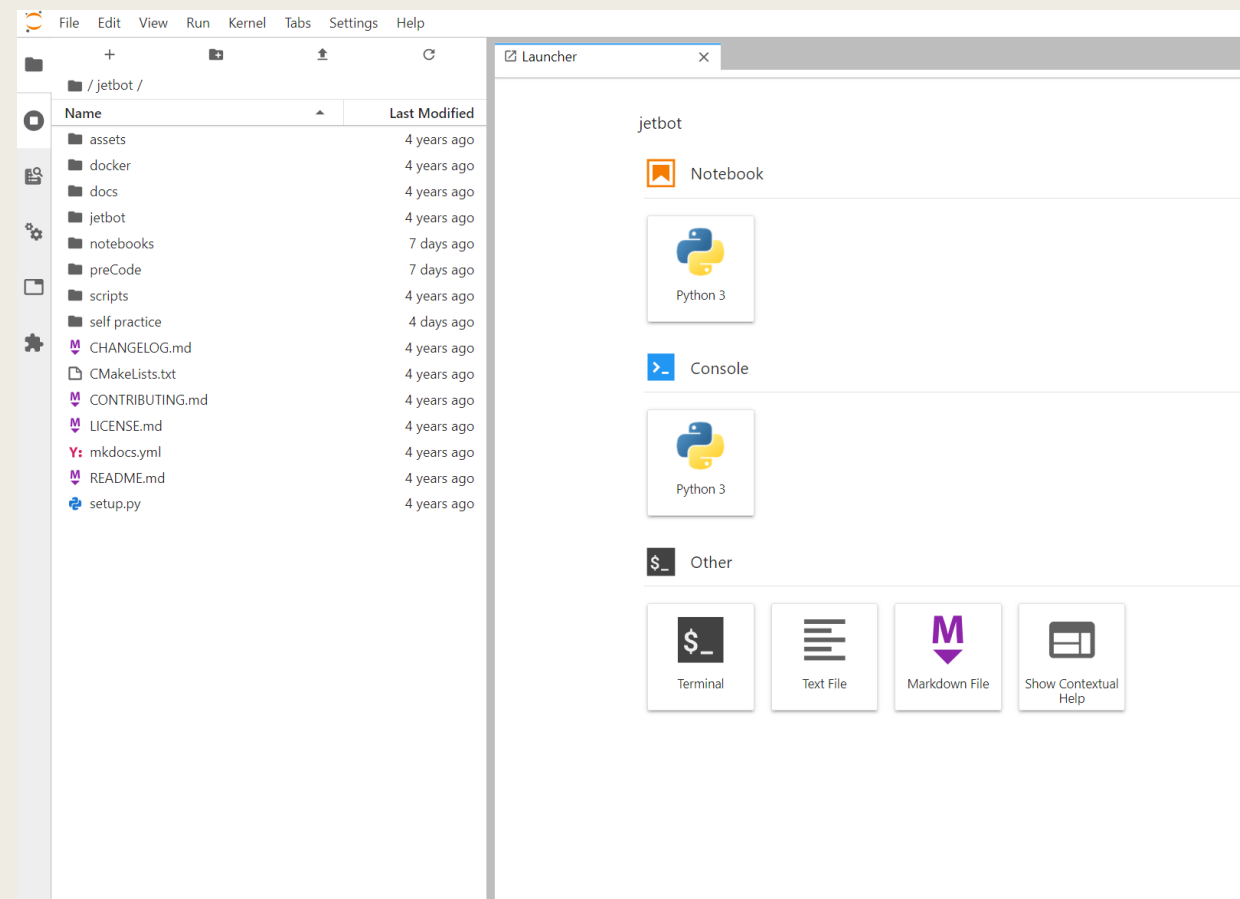
- First, connect your computer and Jetbot to the same network. You can also use your computer's own hotspot.
- After restarting Jetbot, you should see the IP address displayed on the Jetbot's OLED screen.
- Let's assume the IP address is 192.168.137.94.
- Enter `http://192.168.137.94:8888/` in your browser.
- If the webpage connects, you've successfully connected.



Jupyter Intro



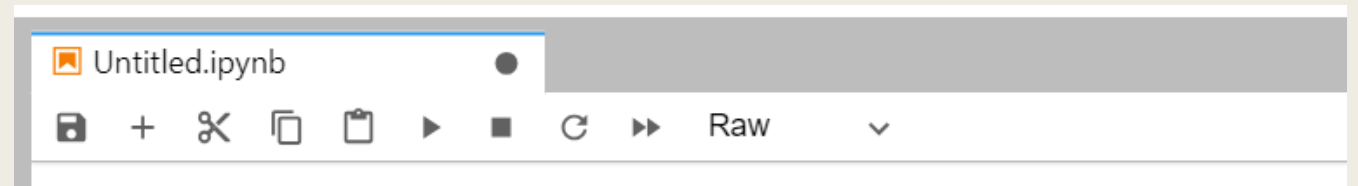
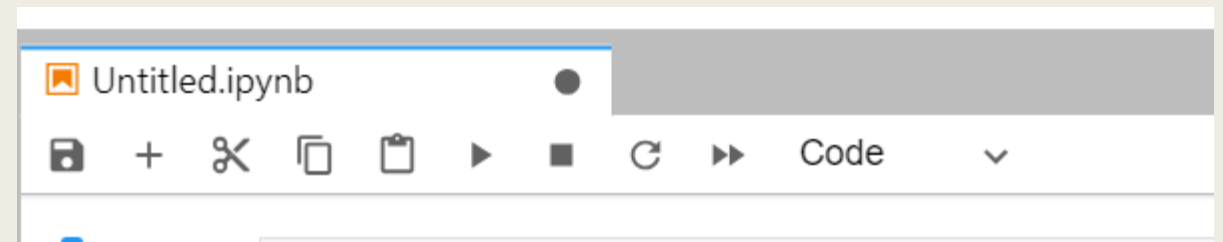
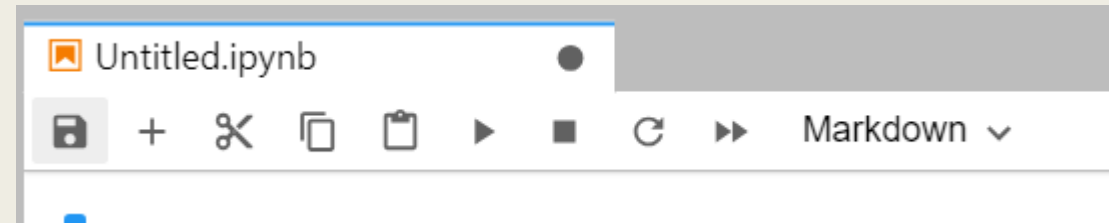
- Jupyter Notebook is a web-based interactive development environment widely used in data science, machine learning, and Python programming. It allows users to write and execute code in a browser, and displays the program's output in real time, including text, images, charts, and more. Jupyter supports multiple programming languages (such as Python, R, and Julia), but Python is the most commonly used.



Basic Jupyter Operations

■ Basic Notebook Structure

- Notebook Cells: The main workspace of a Jupyter Notebook is composed of cells. Each cell can be used to execute Python code or write annotations.
 - **Code Cells:** Used to write and execute code.
 - **Markdown Cells:** Used to write annotations, formulas, or text descriptions. Markdown syntax is supported.
- **Toolbar:** The toolbar allows you to execute cells, stop execution, save the notebook, and perform other operations.



Basic Jupyter Operations

Running Cells

In each "Code Cell," you can write Python code and run it instantly.

After writing code, press Shift+Enter to run the current cell and display the result below the cell.

If the program has output or a chart, it will be displayed on the same page.

Adding or Deleting Cells

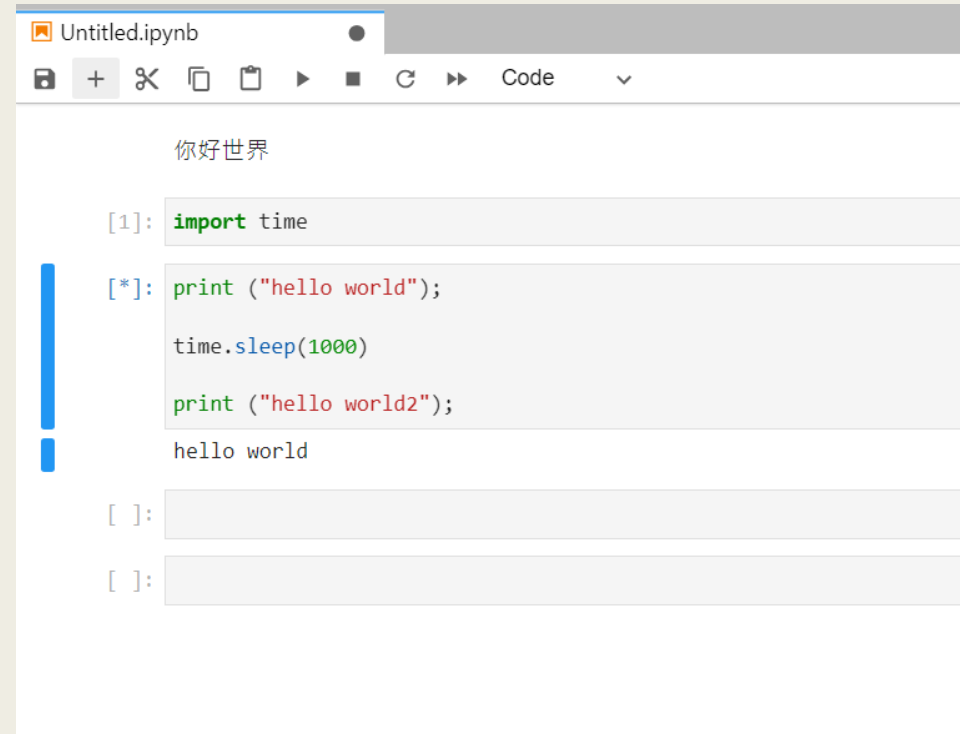
Adding a Cell: Click the "+" button on the toolbar to add a new, blank cell.

Deleting a Cell: Select a cell and click the "scissors" icon to delete the currently selected cell.

Saving and Exporting

Save: Save the current Notebook state by pressing the Save button on the toolbar.

Export: Export your Notebook to various formats (such as HTML and PDF) for easy sharing and publishing.



The screenshot shows a Jupyter Notebook window titled "Untitled.ipynb". The toolbar includes icons for saving, adding a new cell, deleting a cell, and running the cell. The current cell is a code cell containing the following Python code:

```
你好世界

[1]: import time

[*]: print ("hello world");
      time.sleep(1000)
      print ("hello world2");
```

The output of the code cell is displayed below the code:

```
hello world
```

Below the output, there are two empty code cells, each starting with "[]:".

Basic vehicle control-motor test

<code>from jetbot import Robot</code>	Introducing the Jetbot module
<code>robot = Robot()</code>	Declare robot as =Robot
<code>robot.left(speed=0.3)</code>	Turn left 30%
<code>robot.right(speed = 0.3)</code>	Turn right 30%
<code>robot.stop</code>	stop

你好世界

```
: import time|
: print ("hello world");
hello world
: from jetbot import Robot
robot = Robot()
robot.left(speed=0.0)
: robot.stop()
:
```

Basic vehicle control - individual motor control

- `robot.set_motors(0.3, 0.6)` Left side output 30% Right side output 60%
- `robot.left_motor.value = 0.3` Left 30% alone
- `robot.right_motor.value = 0.6` Right 30% alone

Q : Do the following two programs
produce the same result?

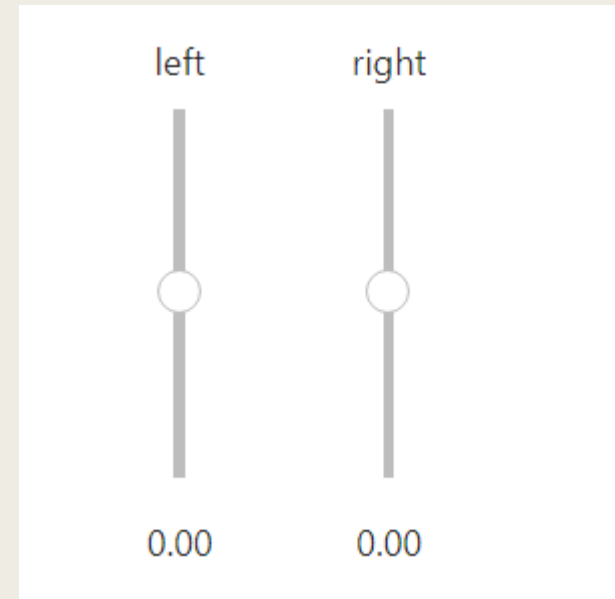
- | | |
|---|--|
| ■ <code>robot.set_motors(0.3, 0.6)</code> | ■ <code>robot.left_motor.value = 0.3</code> |
| ■ <code>time.sleep(1)</code> | ■ <code>robot.right_motor.value = 0.6</code> |
| ■ <code>robot.stop</code> | ■ <code>time.sleep(1)</code> |
| | ■ <code>robot.stop</code> |

Creating a motor control Function

- First limited the input and find the positive and negative FUNCTION
- `def sign(input):`
- `return input / abs(input) if input != 0 else 0`
`# avoid divided by 0`
- `def maxNum(input):`
-
- `if abs(input) > 100:`
- `return 100 * sign(input)`
- `else:`
- `return input`
- `def SpeedControl(s_in,d_in):`
- `L = R = 0`
- `S = maxNum(s_in)`
- `D = maxNum(d_in)*2`
- `L = S+D`
- `R = S-D`
- `L = maxNum(L)`
- `R = maxNum(R)`
- `print(L)`
- `print(R)`

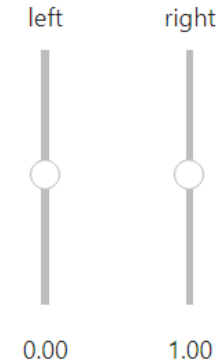
Added on-screen joystick

- `# create two sliders with range [-100, 100]`
- `left_slider = widgets.FloatSlider(description='left', min=-100.0, max=100.0, step=1, orientation='vertical')`
- `right_slider = widgets.FloatSlider(description='right', min=-100.0, max=100.0, step=1, orientation='vertical')`
- `# create a horizontal box container to place the sliders next to each other`
- `slider_container = widgets.HBox([left_slider, right_slider])`
- `# display the container in this cell's output`
- `display(slider_container)`



Update the motor value with the joystick value

- import traitlets
- def update_speed(change):
 - # Get the value of the slider
 - left_value = left_slider.value
 - right_value = right_slider.value
 -
 - # Calculate motor speed using the SpeedControl function
 - L, R = SpeedControl(left_value, right_value)
 -
 - # Updates the displayed motor speed
- # Monitor slider changes and trigger callbacks
- left_slider.observe(update_speed, names='value')
- right_slider.observe(update_speed, names='value')



```
[56]: import traitlets

def update_speed(change):
    # 獲取滑動條的值
    left_value = left_slider.value
    right_value = right_slider.value

    # 使用 SpeedControl 函數計算馬達速度
    L, R = SpeedControl(left_value, right_value)

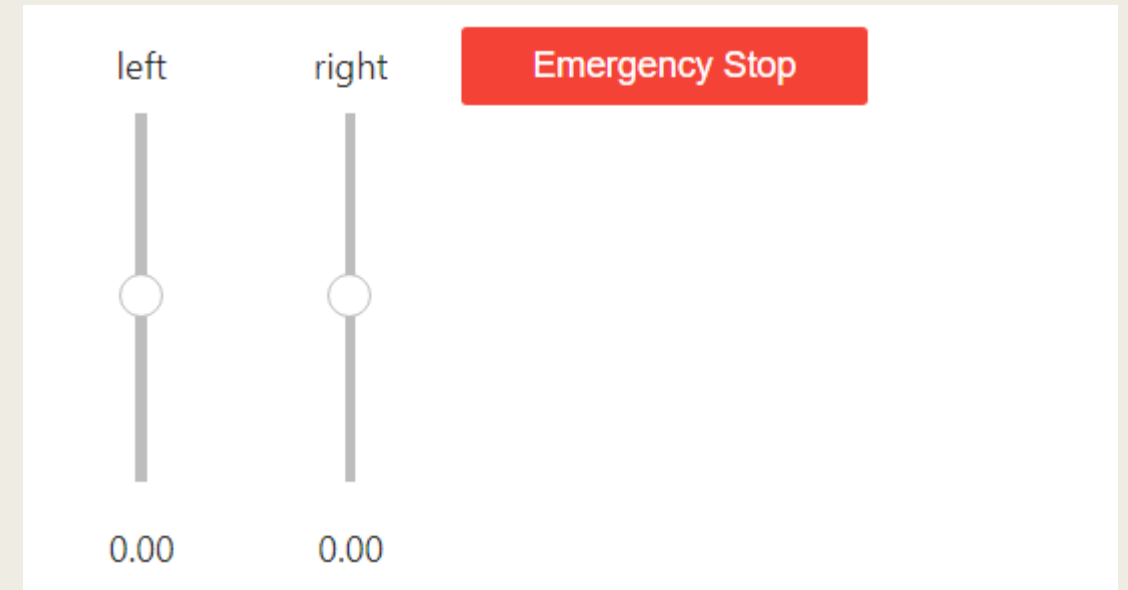
    # 更新顯示的馬達速度

# 監聽滑動條變化並觸發回調
left_slider.observe(update_speed, names='value')
right_slider.observe(update_speed, names='value')
```

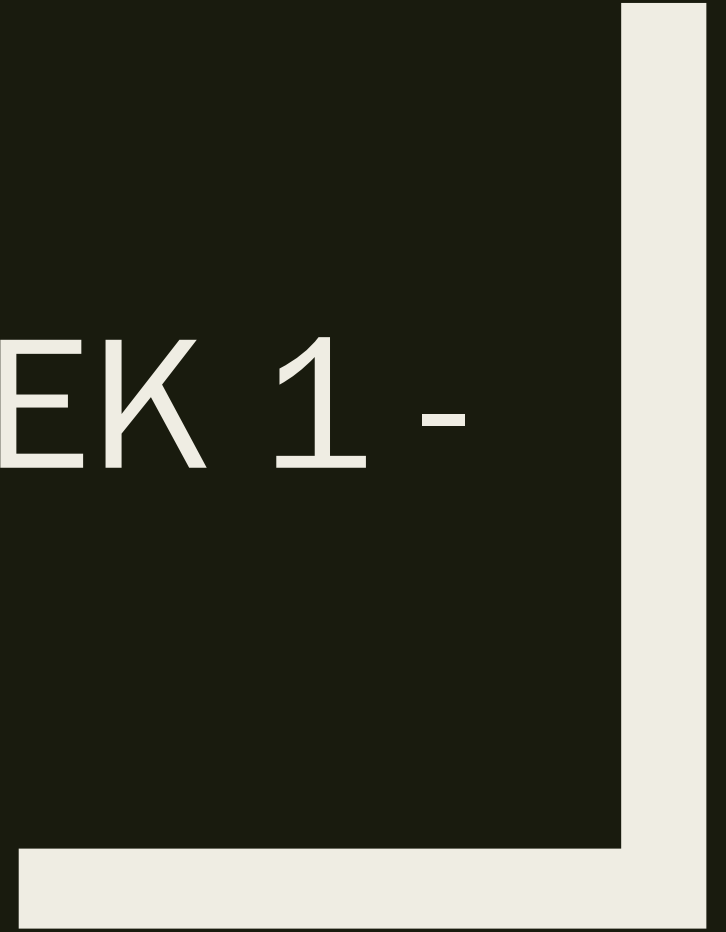
EMS Emergency Stop

- `from jetbot import Heartbeat`
- `heartbeat = Heartbeat()`
- `def handle_heartbeat_status(change):`
 - `if change['new'] == Heartbeat.Status.dead:`
 - `# When the heartbeat stops, stop the robot`
 - `robot.stop()`
- `heartbeat.observe(handle_heartbeat_status, names='status')`
- `def on_button_clicked(b):`
 - `robot.stop()`
- `emergency_button.on_click(on_button_clicked)`

Add a manual button
`emergency_button =`
`widgets.Button(description="Emergency Stop",`
`button_style='danger')`



END OF WEEK 1 -



Test the camera first

- `import ipywidgets.widgets as widgets`
- `from jetbot import Robot`
- `import traitlets`
- `robot = Robot()`
- `image = widgets.Image(format='jpeg', width=300, height=300)`
- `display(image)`
- `from jetbot import Camera`
- `camera = Camera.instance()`
- `from jetbot import bgr8_to_jpeg`
- `camera_link = traitlets.dlink((camera, 'value'), (image, 'value'), transform=bgr8_to_jpeg)`
- `camera.stop()`

```
[11]: import ipywidgets.widgets as widgets
      from jetbot import Robot
      import traitlets

      robot = Robot()

      image = widgets.Image(format='jpeg', width=300, height=300)

      display(image)

      Image(value=b'', format='jpeg', height='300', width='300')

[12]: from jetbot import Camera

      camera = Camera.instance()

[13]: from jetbot import bgr8_to_jpeg

      camera_link = traitlets.dlink((camera, 'value'), (image, 'value'), transform=bgr8_to_jpeg)

[10]: camera.stop()

[ ]:
```

The camera cannot be turned on normally

- 1. The camera cable is connected incorrectly or incorrectly.
- 2. The program is running in the wrong order.
- 3. Repeatedly opening and declaring the camera. **so** `camera.stop()`
- 4. Bad luck.

Normal screen

```
[10]: import ipywidgets.widgets as widgets
      from jetbot import Robot
      import traitlets

      robot = Robot()

      image = widgets.Image(format='jpeg', width=300, height=300)
      display(image)
```



```
[11]: from jetbot import Camera

      camera = Camera.instance()
```

```
[12]: from jetbot import bgr8_to_jpeg

      camera_link = traitlets.dlink((camera, 'value'), (image, 'value'), transform=bgr8_to_jpeg)
```

https://www.waveshare.com/wiki/IMX219-160_Camera

Solve the purple frame problem of the camera

1. Download the camera-override.isp file and extract it to a specific folder:

```
wget https://files.waveshare.com/upload/e/eb/Camera_overrides.tar.gz
tar zxvf Camera_overrides.tar.gz
sudo cp camera_overrides.isp /var/nvidia/nvcam/settings/
```

2. Install files:

```
sudo chmod 664 /var/nvidia/nvcam/settings/camera_overrides.isp
sudo chown root:root /var/nvidia/nvcam/settings/camera_overrides.isp
```

```
wget https://files.waveshare.com/upload/e/eb/Camera_overrides.tar.gz
tar zxvf Camera_overrides.tar.gz
sudo cp camera_overrides.isp /var/nvidia/nvcam/settings/
```

```
sudo chmod 664 /var/nvidia/nvcam/settings/camera_overrides.isp
sudo chown root:root /var/nvidia/nvcam/settings/camera_overrides.isp
```

Do not use jupyter to modify it directly. You must modify it directly on the Jason nano console!!!

